LONG-TERM MEMORY ENGINE FOR AI AGENTS

A PROJECT REPORT

Submitted by

M. KRITHIKRAJ	**2023115004**
ALSTON S	**2023115006**
ARYAN I	**2023115021**
HARI SRINIVASAN	**2023115022**

submitted to the Faculty of

COMPUTER SCIENCE AND INFORMATION ENGINEERING

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

INFORMATION TECHNOLOGY



DEPARTMENT OF INFORMATION SCIENCE AND TECHNOLOGY

COLLEGE OF ENGINEERING, GUINDY

ANNA UNIVERSITY

CHENNAI – 600 025

OCTOBER 2026

ANNA UNIVERSITY

CHENNAI – 600 025

BONA FIDE CERTIFICATE

Certified that this project report titled **LONG-TERM MEMORY ENGINE FOR AI AGENTS** is the bona fide work of **M. KRITHIKRAJ (2023115004), ALSTON S (2023115006), ARYAN I (2023115021)** and **HARI SRINIVASAN (2023115022)** who carried out the project work under my supervision. Certified further that to the best of my knowledge and belief, the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or an award was conferred on an earlier occasion on this or any other candidate.

PLACE: Chennai

DATE:

DR. BAMA SRINIVASAN
PROJECT GUIDE
Department of Information Science and Technology
College of Engineering, Guindy
Anna University, Chennai – 600 025

COUNTERSIGNED

HEAD OF THE DEPARTMENT
Department of Information Science and Technology
College of Engineering, Guindy
Anna University, Chennai – 600 025

ABSTRACT

AI agents that don't have persistent memory forget instructions and project decisions once a session ends, and when an instruction changes they often keep using the old one. For this project we built a long-term memory engine in Rust for such agents. What it remembers carries over between sessions, can be corrected later, and can be traced back to the original messages.

The engine has five modules: evidence acquisition and processing, knowledge extraction and consolidation, temporal knowledge and graph management, hybrid retrieval and grounded reasoning, and model training and evaluation. Original messages are stored unchanged. Facts are extracted along with the exact quote they came from and stored as versions, so a correction replaces an old value without deleting it. Retrieval combines keyword, vector, time and graph search using Reciprocal Rank Fusion and returns cited evidence. The extraction and answering is done by a small local Qwen3-4B model that we fine-tuned with QLoRA.

M1 is implemented, M2 to M4 are implemented with some quality and benchmark work left, and the M5 model is trained and integrated while evaluation is still going on. So far all 142 stored source quotes matched their original text, and fine-tuning improved combined facts from 1/16 to 13/16 and answers from 10/16 to 15/16. On the other hand fact coverage dropped from 50/55 to 41/55 and dates are still handled poorly. The full LongMemEval-S run is pending.

ACKNOWLEDGEMENT

We express our sincere gratitude to our guide, Dr. Bama Srinivasan, Department of Information Science and Technology, College of Engineering, Guindy, Anna University, for her guidance, regular reviews and constructive suggestions throughout this project. We also thank the Department of Information Science and Technology for the facilities and support provided.

We acknowledge the open-source and research community whose published work and software made this project possible, including the authors of the systems and papers cited in this report.

M. KRITHIKRAJ
ALSTON S
ARYAN I
HARI SRINIVASAN

TABLE OF CONTENTS

ABSTRACT iv
LIST OF TABLES viii
LIST OF FIGURES ix
LIST OF ABBREVIATIONS x
GLOSSARY xi

1 INTRODUCTION **1**
1.1 BACKGROUND 1
1.2 PROBLEM STATEMENT 2
1.3 OBJECTIVES 2
1.4 CHALLENGES FACED 3
1.5 ORGANISATION OF THE REPORT 3

2 LITERATURE SURVEY **4**
2.1 LONG CONTEXT AND EXTERNAL MEMORY 4
2.2 MEMORY SYSTEMS 4
2.3 METHODS USED IN THIS PROJECT 6
2.4 COMPARISON AND GAP 6

3 SYSTEM DESIGN **7**
3.1 DESIGN REQUIREMENTS 7
3.2 OVERALL ARCHITECTURE 7
3.3 M1 EVIDENCE ACQUISITION AND PROCESSING 8
3.4 M2 KNOWLEDGE EXTRACTION AND CONSOLIDATION 9
3.5 M3 TEMPORAL KNOWLEDGE AND GRAPH MANAGEMENT 11
3.6 M4 HYBRID RETRIEVAL AND GROUNDED REASONING 12
3.7 M5 MODEL TRAINING AND EVALUATION 13

4 IMPLEMENTATION **15**
4.1 IMPLEMENTATION ENVIRONMENT 15
4.2 ALGORITHMS 15
4.2.1 Priority-Based Context Selection 16
4.2.2 Entity Resolution and Memory Versioning 16
4.2.3 Memory Consolidation and Invalidation 17

4.2.4 Hybrid Retrieval and Graph Traversal 18
4.2.5 Reciprocal Rank Fusion and Context Packing 18
4.2.6 Evidence-Based Answer Generation 20
4.3 INTEGRATION WITH THE PI AGENT 20

5 DATASET, FINE-TUNING, RESULTS AND STATUS 21
5.1 DATASETS 21
5.2 FINE-TUNING AND MODEL SELECTION 22
5.3 LIVE DEMONSTRATION 23
5.4 RESULTS AND OBSERVATIONS 25
5.5 STATUS AND PENDING WORK 27
5.6 CONCLUSION AND FUTURE WORK 27

REFERENCES 29

LIST OF TABLES

2.1 Comparison of existing memory systems 6

5.1 Qwen3-4B before and after fine-tuning 26

LIST OF FIGURES

3.1 Overall system architecture 8
3.2 Module 1: Evidence Acquisition and Processing 9
3.3 Module 2: Knowledge Extraction and Consolidation 10
3.4 Module 3: Temporal Knowledge and Graph Management 12
3.5 Module 4: Hybrid Retrieval and Grounded Reasoning 13
3.6 Module 5: Model Training and Evaluation 14

5.1 General-purpose training rows across the 16 domains 22
5.2 Demo: recording facts, then recalling them in a new session 24
5.3 Demo: recalled answer and the corrected fact in the graph 25

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
CPU	Central Processing Unit
DMR	Deep Memory Retrieval
GGUF	GPT-Generated Unified Format
HTTP	Hypertext Transfer Protocol
ID	Identifier
JSON	JavaScript Object Notation
LLM	Large Language Model
LoRA	Low-Rank Adaptation
MVCC	Multi-Version Concurrency Control
NF4	4-bit NormalFloat
p95	95th percentile
QLoRA	Quantised Low-Rank Adaptation
RRF	Reciprocal Rank Fusion
SHA-256	Secure Hash Algorithm, 256-bit
SLM	Small Language Model
SQL	Structured Query Language

GLOSSARY

Some everyday computing words are used with a narrower meaning in this report. The meanings below are the ones we use throughout.

Memory	Knowledge about a user or project that the agent keeps across sessions (facts, observations and the messages they came from). It does *not* mean RAM, cache, disk or any other computer memory.
Long-term memory	Memory that survives after a session ends, as opposed to what is only in the current prompt.
Agent	An AI program (here, the Pi coding agent) that talks to the user and carries out tasks using a language model and tools.
Harness	The program around the model that runs the agent: it builds prompts, calls tools and in our case talks to the memory service.
Session	One continuous conversation between the user and the agent. A new session starts with an empty prompt.
Context / context window	The text the model can read at one time (its prompt). Its size is limited and measured in tokens.
Token	A small piece of text (a word or part of a word) that the model reads. Budgets and limits are counted in tokens.
Namespace	A label such as `user:maya` that keeps one user's or project's memory separate from another's.
Evidence	The original messages exactly as they were sent, stored before anything is extracted from them.
Episode	One batch of messages saved together by a single retain request.

Event	A single message (with role and timestamp) inside an episode.
Chunk	A piece of an episode's text that is stored and embedded for search.
Spool	A local folder of files on the client where batches wait if the memory service can't be reached.
Job / worker	A job is a stored task (e.g. "extract facts from this episode"); a worker is a background process that picks up and runs jobs.
Lease	A time-limited lock a worker takes on a namespace so that two workers don't process it at the same time.
Claim	A fact proposed by the model before the system has checked and stored it.
Fact / assertion	A stored statement about an entity, e.g. "Maya lives in Bengaluru", with its source quote and validity.
Entity	A person, project, place or thing that facts are about.
Observation	A higher-level statement built from at least two supporting facts.
Consolidation	The step that combines related facts into an observation.
Invalidation	Marking an observation stale when one of the facts it depends on changes.
Supersede	To replace a fact's value with a newer one while keeping the old fact as history.
Validity interval	The time range during which a fact was true, e.g. Chennai from 1 to 10 October.
Embedding / vector	A list of numbers representing the meaning of a text, used to find similar texts.
Retrieval / recall	Finding the stored memories relevant to a question and returning them to the agent.

Projection	The copy of facts and entities kept in Neo4j as a graph. It can be rebuilt from PostgreSQL.
Reflection	An optional step where the model writes an answer from retrieved memories, with citations.
Grounded answer	An answer in which every statement is backed by a cited, retrieved memory.
Fine-tuning	Further training of an existing model on our own data so it does a specific task better.
Adapter	The small set of extra weights trained by LoRA/QLoRA; the base model itself stays frozen.
Quantisation	Storing model weights with fewer bits (e.g. 4-bit or 8-bit) so the model uses less memory and runs faster. Here "memory" does mean computer memory.
Teacher model	A larger model (GPT-5.6-sol) used to write the expected outputs for our training data.

CHAPTER 1

INTRODUCTION

AI agents are now used for coding projects that go on for weeks over many sessions, and every new session starts with an empty prompt. Whatever the user told the agent last week is simply gone unless the user types it in again. This project gives the agent a long-term memory outside the model.

1.1 BACKGROUND

The obvious fix is to paste the whole history into the prompt every time. This does not work well. Liu et al. [1] showed that language models use long inputs unevenly and miss details that sit in the middle of the context. Old statements also sit right next to their corrections with no rule for which one is current.

So a memory component has to decide what to keep and what to give back for a question. We built this as a separate service for a coding agent (Pi [9]). It stores the original messages, pulls facts out of them, remembers when a fact changed, and when the agent asks, returns only the relevant facts along with the quote they came from.

The word *memory* in this report always means this kind of stored knowledge about the user and the project, not RAM or any other computer memory. Since a few other terms are also used in a specific sense, they are listed in the Glossary.

The engine has five modules: M1 saves the original messages

(Evidence Acquisition and Processing), M2 pulls facts out of them (Knowledge Extraction and Consolidation), M3 stores the facts with their history (Temporal Knowledge and Graph Management), M4 finds the right facts for a question (Hybrid Retrieval and Grounded Reasoning) and M5 trains the small local model the others use (Model Training and Evaluation). M1 to M4 make up the memory service and M5 is an offline pipeline.

1.2 PROBLEM STATEMENT

Agents without persistent memory lose instructions and project decisions when a task continues across sessions, and when an instruction changes they may reuse the old value. Users end up repeating context and fixing the same mistakes again.

Throughout the report we use one small example, a user called Maya. On 1 October 2026 she tells the agent "My name is Maya. I live in Chennai. I work as a software engineer at NovaTech, and I work remotely." (we call this message E1). On 10 October she says she has moved to Bengaluru (E3). On 13 October she asks what the agent remembers about where she lives, her previous city and her work (E5).

An agent with no memory can't answer E5, and one that only kept E1 would confidently say Chennai, which is worse. We want it to know Bengaluru replaced Chennai on 10 October and still keep Chennai as her previous city.

1.3 OBJECTIVES

1. Retain useful information for AI agents across sessions.
2. Keep the updated instruction while preserving the earlier versions.

3. Link every stored fact back to the original message so it can be verified.
4. Fine-tune a small local model to extract facts, combine related facts and answer from evidence.
5. Retrieve relevant information quickly within a limited context size.
6. Evaluate memory accuracy and retrieval with repeatable tests, including the LongMemEval-S benchmark [2].

1.4 CHALLENGES FACED

On the fine-tuning side, our early models did well on generated test data but struggled on real conversations (only 18 of 33 conversation segments passed the extraction checks). The models also mixed up quotations, source references and dates. We revised the data, prompts and validation several times. One issue we have not solved: fine-tuning made combining facts and answering questions much better, but fact coverage dropped from 50/55 to 41/55.

On the integration side the model sometimes re-extracted an outdated value, so correction handling had to be reworked. Repeated messages made duplicates and one failed job could block the rest. The hardest bug to notice was that the deployed model silently cut off inputs longer than its limit, so now we check input length and split long conversations into smaller pieces.

1.5 ORGANISATION OF THE REPORT

Chapter 2 reviews existing memory systems, Chapter 3 the architecture and modules, Chapter 4 the tools and algorithms, and Chapter 5 the data, fine-tuning, demo and results.

CHAPTER 2

LITERATURE SURVEY

This chapter looks at existing work on long-term memory for agents. Every number quoted here is the one reported by the authors on their own setup; we did not rerun any of these systems.

2.1 LONG CONTEXT AND EXTERNAL MEMORY

The simplest memory is to keep the entire conversation in the prompt. Liu et al. [1] found that models use information in the middle of a long context less reliably than information near the start or end, so a bigger context window does not remove the need to choose what the model reads. It is also slow (17 s p95 latency for the full history in the Mem0 paper).

All the systems below handle this the same basic way. They keep memory outside the prompt and only put a selected part of it in front of the model. Where they differ is what they store, who decides how a stored item changes (the model or the code), and whether the old value of a changed fact is kept.

2.2 MEMORY SYSTEMS

MemGPT. Packer et al. [3] split the prompt into a main context and external recall and archival storage, and let the model move data between them using function calls. On deep memory retrieval they report 92.5% for GPT-4 with MemGPT against 32.1% for GPT-4 reading a summary. A missed function call means a missed update.

Zep and Graphiti. Rasmussen et al. [4] store a temporal knowledge graph of episodes, entities and facts. Each fact has a valid time and an invalid time, and a contradicting fact closes the old one instead of deleting it. They report 71.2% on LongMemEval with gpt-4o against 60.2% for the full 115k-token history, from a single run. This is closest to ours, except that in Zep the graph is the main store while we keep the real data in PostgreSQL.

Mem0. In Mem0 [5] an LLM extracts candidate facts, compares each one with the nearest stored memories and picks one of add, update, delete or no-op. On LoCoMo they report a judge score of 66.9 against 72.9 for the full history, with p95 latency of 1.4 s against 17 s. Only one benchmark and an unnamed judge model were used, and the open-source release has since dropped update, delete and graph memory. We use the same extract-then-compare idea, but in our system the update decision is made by fixed code rules and not by the LLM for each fact.

Supermemory. Supermemory [6] stores atomic memories linked to source chunks, with "updates", "extends" and "derives" links and two timestamps on each. Its results are only from the vendor's own report and the engine is closed source.

Hindsight. Hindsight [7] is built around three operations, retain, recall and reflect. Recall combines four retrieval strategies using rank fusion and packs the result into a token budget. This directly influenced our recall step. It is not peer reviewed though.

Benchmarks. LongMemEval [2] tests chat assistants on long-term interactive memory, and Zep, Hindsight and Supermemory all report on it. LoCoMo [15] evaluates very long-term conversational memory and is the only benchmark used in the Mem0 paper. The published results aren't really

comparable since each uses its own models and judge.

2.3 METHODS USED IN THIS PROJECT

We use a few well known methods as building blocks. PostgreSQL full-text search [10] with GIN indexes is our keyword search. pgvector [11] adds vector columns and nearest-neighbour search to the same database, which we use with cosine distance on 1,024-dimensional vectors. Reciprocal Rank Fusion [12] merges several ranked lists by adding up $1/(k+rank)$ for each item; we use $k = 60$. The vectors come from a Qwen3 Embedding model [13] served through Ollama, and the memory model is fine-tuned with QLoRA [14], which trains small adapter weights on top of a frozen, quantised base model.

2.4 COMPARISON AND GAP

Table 2.1: Comparison of existing memory systems

System	Who decides an update	Main limitation
MemGPT	The model, through function calls	Edits depend on correct calls; no cost figures
Zep / Graphiti	Graph rule (validity intervals)	Single run; graph database is the main store
Mem0	The LLM, per fact	One benchmark; released code differs from paper
Supermemory	Not fully documented	Closed source, vendor report only
Hindsight	Retain step	Not peer reviewed; budget not stated

Table 2.1 puts the five systems side by side. In all of them the model decides updates or the rule is tied to a graph database. So we took the extract-then-compare idea, the validity intervals and the fused recall, but made the update rule plain service code, kept PostgreSQL as the source of truth, tied every fact to an exact quote, and trained a small local extraction model.

CHAPTER 3

SYSTEM DESIGN

This chapter describes the design requirements, the architecture and the five modules, following the Maya example through them.

3.1 DESIGN REQUIREMENTS

We worked from a few rules. Original messages are saved exactly as sent *before* any extraction, which runs later as a queued job, so a failed model call can't lose a conversation. A changed fact is a new version and the old value is kept with its validity interval, never overwritten. Every fact links to an exact quote that is checked against the original message. All data belongs to a namespace (e.g. `user:maya`) and whatever we hand to a model or the agent has to fit a token budget. Models run locally on Ollama. Duplicates, crashed workers and bad model output are handled with stored jobs, leases, retries and validation, and Neo4j is only a copy that can be rebuilt from PostgreSQL.

3.2 OVERALL ARCHITECTURE

Figure 3.1 shows the overall architecture. The client is the Pi coding agent. Its memory client saves every message through the memory service and asks for recall before answering. The memory service itself is a Rust application with API handlers and asynchronous workers, and it depends on four other components.

PostgreSQL with pgvector is the main store, shared by M1 to M4. It holds the evidence (sessions, episodes, events, chunks), the entities, assertions,

sources and vectors, and the job queue. Ollama runs the models: our fine-tuned worker model for M2 (and reflection in M4), and a Qwen3 embedding model for vectors. Neo4j keeps a graph copy of the assertions and entities, joined by `MENTIONS` and `SUPPORTS` edges. M4 rechecks every graph hit against PostgreSQL before using it. M5 runs offline (live chats are *not* used as training data).

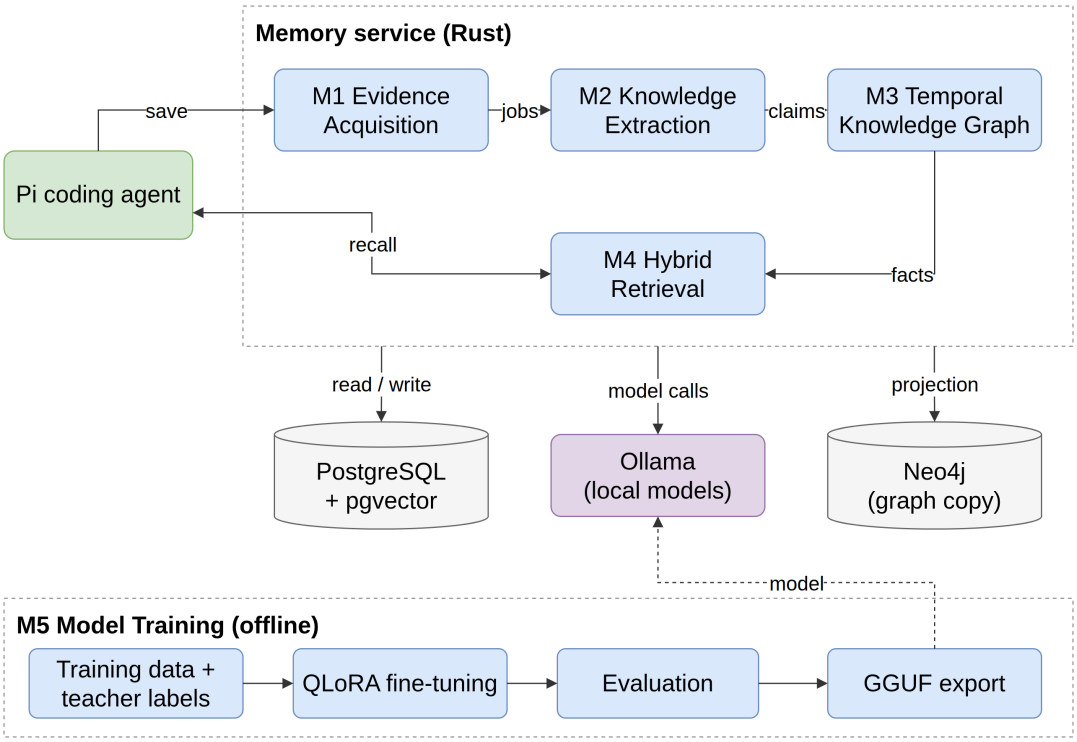


Figure 3.1: Overall system architecture

3.3 M1 EVIDENCE ACQUISITION AND PROCESSING

M1 saves the evidence before anything else (Figure 3.2). For Maya, the 10 October conversation arrives as a retain request with namespace `user:maya`, session S2 and the two events E3 (her message) and E4 (the agent's reply).

The evidence collector inside Pi gathers the role, text and timestamp of each message on the current branch and forms a `RetainBatch`. This batch

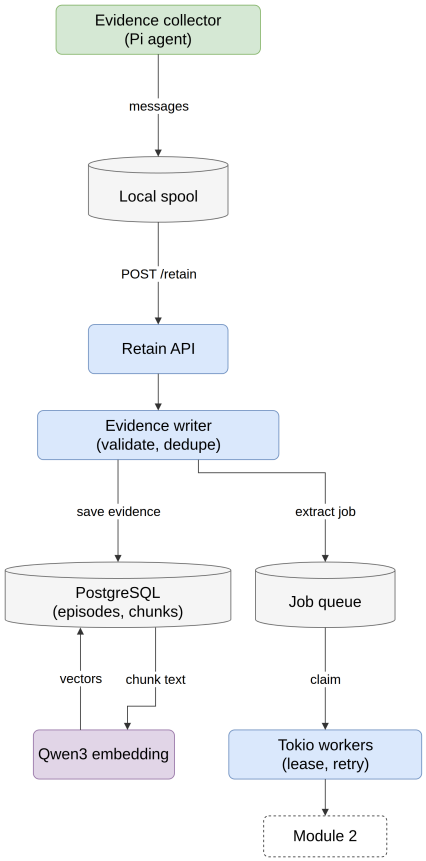


Figure 3.2: Module 1: Evidence Acquisition and Processing

is first written to a local spool of JSON files so nothing is lost if the service is down, and the spool then sends it to `POST /api/v2/retain`. The writer checks the batch (1 to 1,000 events). If the same key arrives again with the same SHA-256 hash it just returns the existing IDs, so retries don't create duplicates; the same key with a different payload gets HTTP 409. The episode, events and chunks are saved in one transaction together with a pending extraction job in `memory_jobs`. Each chunk is also embedded into a 1,024-dimensional vector using `qwen3-embedding-cpu`.

Jobs are picked up by a pool of Tokio workers. A worker leases the namespace for 15 minutes (renewed every 30 s) and a job gets up to 5 tries with backoff up to 300 s. We run two workers locally (the default is four). For Maya, M1 ends with episode P2 holding chunks C3 and C4, and job J2 pending.

3.4 M2 KNOWLEDGE EXTRACTION AND CONSOLIDATION

M2 has two stages, extraction and consolidation, and both call the same local model `mem-extractor:general-v1` through Ollama (Figure 3.3). Outputs are cached in `model_cache`.

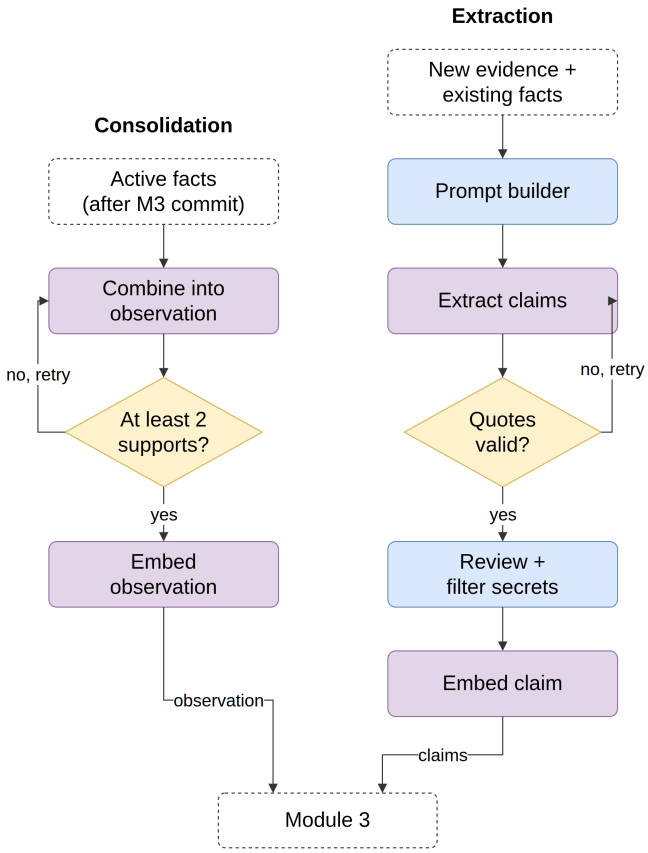


Figure 3.3: Module 2: Knowledge Extraction and Consolidation

Extraction. The prompt builder fits the new evidence and the relevant existing facts into the model's input (how existing facts are chosen is in Section 4.2.1). For Maya the existing fact "Maya / Chennai" is shown next to E3. The extractor returns claims with exact quotes. Here it returns home city Bengaluru, single-valued and flagged as a correction. The output is checked against the schema and each quote must appear exactly in the source; if not, the model is asked again up to two times before the job fails. A review step confirms the correction, secrets are filtered out, and the claim "Maya lives in

Bengaluru.” goes to M3.

Consolidation. After M3 commits, M2 looks at the active facts of the changed subject. If there are at least two, the model may combine them into one observation. For Maya, the three work facts A1 (NovaTech), A2 (software engineer) and A3 (remote) become “Maya works remotely as a software engineer at NovaTech.”

3.5 M3 TEMPORAL KNOWLEDGE AND GRAPH MANAGEMENT

M3 is the only module that writes the actual facts (assertions). Everything for one job happens inside a single PostgreSQL transaction that holds a namespace lock and checks the job’s lease (Figure 3.4).

For a claim, M3 first resolves the entity (Maya, a person in `user:maya`). If the same value already exists, it is just reinforced. If the slot is empty, a new active assertion is made. The interesting case is a different value in a single-valued slot, like Chennai to Bengaluru. If the claim is a reviewed correction with confidence at least 0.8, the old assertion is superseded and the new one becomes active; otherwise both are marked contested. Maya’s claim has confidence 0.95, so R1 Chennai becomes valid from 1 to 10 October and superseded, and R2 Bengaluru is active from 10 October, linked to the E3 quote.

After the transaction commits, a projection worker copies the changed assertion into Neo4j over HTTP [8]. These writes are revision-checked so an old revision can’t overwrite a newer one.

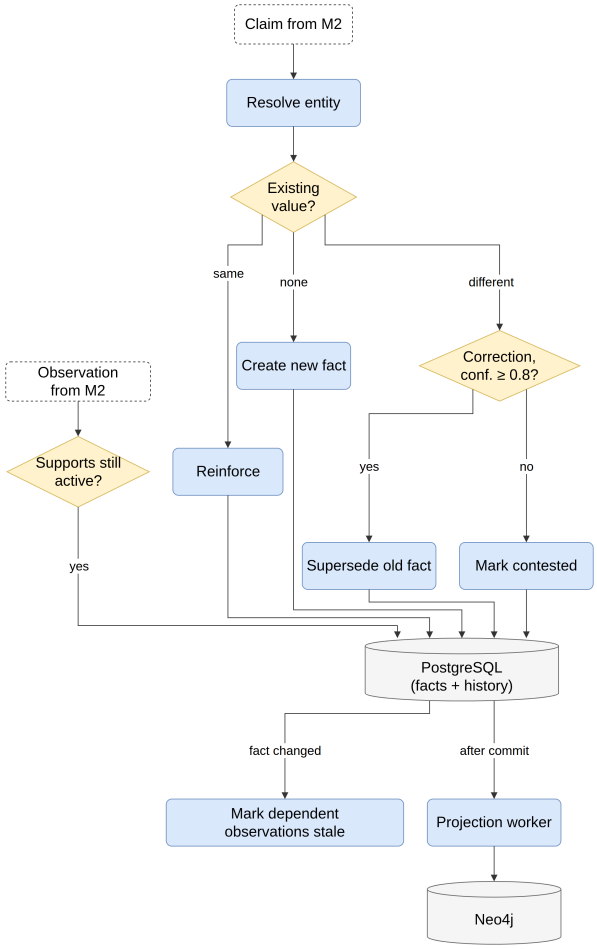


Figure 3.4: Module 3: Temporal Knowledge and Graph Management

3.6 M4 HYBRID RETRIEVAL AND GROUNDED REASONING

M4 answers recall requests (Figure 3.5). On 13 October Pi sends E5 with namespace `user:maya`, `top_k` 10, a maximum distance of 0.45 and a 2,048 token budget.

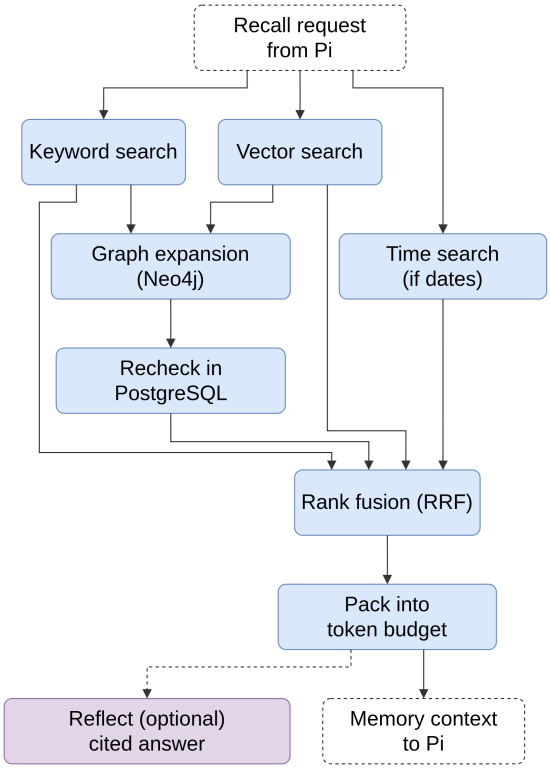


Figure 3.5: Module 4: Hybrid Retrieval and Grounded Reasoning

The query goes to keyword, vector and (if there's a date window) time-based search, the top results seed a Neo4j expansion, and the lists are merged with Reciprocal Rank Fusion and packed into the budget with up to three quotes per hit. There is also an optional reflect endpoint that writes a cited answer. All of this is in Chapter 4.

3.7 M5 MODEL TRAINING AND EVALUATION

M5 runs offline and produces the worker model used by M2 and M4 (Figure 3.6).

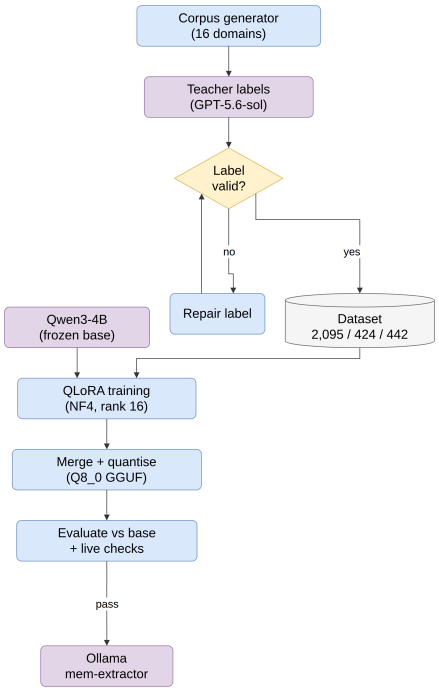


Figure 3.6: Module 5: Model Training and Evaluation

A corpus generator makes short episodes across 16 domains (the Maya episode is one of them) and a teacher model (GPT-5.6-sol) labels each one. Checked labels form the dataset, and the model is trained with QLoRA, evaluated and imported into Ollama (details in Chapter 5).

So when Maya asks E5, Pi answers “You now live in Bengaluru, following your move on 10 October 2026. Before that, you lived in Chennai. You work remotely as a software engineer at NovaTech.”

CHAPTER 4

IMPLEMENTATION

This chapter covers the tools we used, the six main algorithms (each with the Maya example walked through it) and how the engine plugs into Pi.

4.1 IMPLEMENTATION ENVIRONMENT

The memory service is written in Rust. Tokio gives us the async runtime for the background workers, Axum handles the API requests and SQLx talks to PostgreSQL. PostgreSQL 17 with pgvector is the main store and handles both full-text and vector search, and Neo4j 5.26 holds the graph copy used for graph-based retrieval.

The models run locally on Ollama. Our extraction model is a fine-tuned Qwen3-4B-Instruct-2507, which extracts facts, combines them and answers from evidence, and Qwen3-Embedding-0.6B makes the vectors. On the agent side we use the Pi agent harness to capture messages and inject memory. The dashboard (session controls and the memory graph) is React + TypeScript with Vite and Bootstrap.

Training used Python, PyTorch, Transformers, PEFT and bitsandbytes, with llama.cpp to make the GGUF. Docker Compose runs the services, and Caddy and ttyd give browser access to the app and the Pi console.

4.2 ALGORITHMS

Algorithm 4.1 Priority-based context selection

```
Input: existing facts, new messages, available input space

IF all existing facts fit:
    RETURN all facts
Score facts by subject and value mentions in new messages
Rank stronger matches first; use recency to break ties
Use binary search to find the largest ranked subset that
    fits
RETURN selected facts in their original order
```

There are six algorithms. Context selection is in M2; entity resolution and versioning is in M3; consolidation and invalidation is split between M2 (making observations) and M3 (marking them stale); and retrieval, rank fusion with packing, and answer generation are all in M4.

4.2.1 Priority-Based Context Selection

The extraction model needs to see the facts already stored about the same people, otherwise it can't tell a repeat from a correction, but they may not all fit its input limit. So facts mentioned in the new messages are ranked first (recency breaks ties) and a binary search finds the largest prefix that fits.

For Maya, when E3 ("I moved from Chennai to Bengaluru today…") comes in, she has four stored facts: Chennai, NovaTech, software engineer and remote. Assume for the example that only three fit. The home city fact matches E3 on both subject and value, so it ranks first and the other three follow by recency. Binary search settles on three facts, the home city one included, and these go to the extractor with E3.

4.2.2 Entity Resolution and Memory Versioning

Algorithm 4.2 Entity resolution and memory versioning

```
Input: a new fact and stored memories

Match entity by type and normalised name, or a unique
    stored alias; otherwise create an entity
IF the same fact exists:
    Attach the new evidence
ELSE IF no conflicting single-value fact exists:
    Store the new fact as active
ELSE IF explicit correction AND confidence >= 0.8:
    Supersede the old fact; activate the new fact
ELSE:
    Mark both values as disputed
```

This decides which entity a claim is about and what it does to the stored facts. The important rule is that a conflicting value never overwrites silently, and the old fact is superseded, not deleted, which is what lets us answer the "previous city" part of E5.

The E3 claim is Maya, home city Bengaluru, single-valued, `correction=true`, confidence 0.95, quote "Bengaluru is now my home city." Maya is found, so no new entity. The same fact doesn't exist (R1 says Chennai), there is a conflicting single-valued fact, and the claim is a correction with 0.95 $\geq$ 0.8. So R1 Chennai is superseded with validity 1–10 Oct, and R2 Bengaluru is active from 10 Oct with E3 as its source.

4.2.3 Memory Consolidation and Invalidation

The model proposes an observation but the code decides whether to save it. Supports are rechecked just before saving since they may have changed while the model was running. Invalidation goes the other way and marks dependent observations stale.

Algorithm 4.3 Memory consolidation and invalidation

```
Input: active facts about the affected entities

Ask the Extractor SLM to combine related facts into
    observations
Require at least two distinct supporting facts
Check support IDs and the observation's subject
Recheck that supporting facts are active before saving
Store each observation with its evidence links

WHEN a supporting fact changes or is retracted:
    Recursively find dependent observations
    Mark them as stale
```

For Maya the inputs are A1 (NovaTech), A2 (software engineer) and A3 (remote). The model proposes "Maya works remotely as a software engineer at NovaTech." with all three as supports. It has three distinct supports, the IDs exist, the subject is Maya, and all three are still active, so it is saved as O1. If A3 were retracted later, O1 would be found as a dependent and marked stale.

4.2.4 Hybrid Retrieval and Graph Traversal

This returns separate ranked candidate lists. The graph expansion is limited to two rounds so it doesn't pull in the whole graph. Merging is left to the next algorithm.

E5 has no date phrase, so there is no time-based list. Keyword and vector search find R2 (Bengaluru) and the work facts. The graph step starts from these and the Maya entity: `MENTIONS` reaches Maya, and the `supersedes` link of R2 reaches R1 (Chennai). The graph hits are rechecked in PostgreSQL for status, time and distance, and the lexical, semantic and graph lists are passed on.

Algorithm 4.4 Hybrid retrieval and graph traversal

```
Input: a question and optional date restrictions

Resolve supported date phrases when planning is enabled
Find candidates using keyword and vector searches
Include time-based candidates when a date window exists
Start graph traversal from matching facts and entities
Expand through entity/support links for two bounded rounds
Filter results by validity and any supplied relevance
    cutoff
RETURN candidate lists for ranking
```

Algorithm 4.5 Reciprocal Rank Fusion and context packing

```
Input: ranked candidate lists and a context budget

FOR each memory:
    Score = SUM[1 / (60 + rank)] across search lists
Sort by decreasing score; break ties by memory ID
FOR each ranked memory:
    Build a block containing the fact and source evidence
    Add it if it fits; otherwise consider the next memory
    Stop when the requested number of memories is reached
RETURN the packed context
```

4.2.5 Reciprocal Rank Fusion and Context Packing

Scores from different searches can't be compared, so only ranks are used. A memory that several lists rank highly beats one that a single list ranks first, and ties go by ID so the order is deterministic.

To illustrate, say R2 is ranked first and R1 second in the lexical, semantic and graph lists. Then R2 scores $3 \times 1/61 \approx 0.0492$ and R1 scores $3 \times 1/62 \approx 0.0484$, so R2 comes first and no tie-break is needed. With Pi's settings (`top_k` 10, 2,048 tokens) both blocks fit easily, and R2 goes out with

Algorithm 4.6 Evidence-based answer generation

```
Input: a question and retrieved memories

IF no memories were retrieved:
    RETURN "Insufficient evidence."
Ask the extractor SLM to answer from the retrieved evidence
Restrict citation IDs to the retrieved memories
Check the response format and citation IDs
Require citations when sufficient evidence is claimed
RETURN the answer, or reject invalid output
```

the E3 quote attached.

4.2.6 Evidence-Based Answer Generation

This decides whether the model's answer is accepted. Bad output is rejected rather than patched up, so every citation the user sees points to a memory that was actually retrieved.

For E5 the evidence is R2, R1 and the work facts, so the model is called. Only R1, R2 and the work fact IDs are allowed as citations; the answer cites R2 for Bengaluru and R1 for Chennai, so it passes.

4.3 INTEGRATION WITH THE PI AGENT

Pi's memory client saves every message, tagged with a namespace and session. When Maya asks E5, Pi sends a recall request for `user:maya` with `include_raw` true, `max_distance` 0.45, `top_k` 10 and `max_tokens` 2,048. The result is injected only if non-empty and from the same namespace.

CHAPTER 5

DATASET, FINE-TUNING, RESULTS AND STATUS

This chapter covers the data, the fine-tuning, a demo, our results so far and what is left to do.

5.1 DATASETS

Each training row has the same shape: the input is a short conversation plus a task (extract, consolidate or reflect), and the output is the expected memory, i.e. facts, observations or an answer, each tied to a source quote. For example in the Maya episode the user says "I live in Chennai." and later "Bengaluru is now my home city.", and the expected output is the fact "Maya, home city, Bengaluru" with `correction:true`, `source:user` and that quote.

The fine-tuning data started out as only coding conversations and we later expanded it to 16 domains (family, food, travel, education, work, software and so on). All conversations are synthetic. They were generated by the teacher model GPT-5.6-sol, cross-checked by a second adversarial model and then cleaned. In total there are 2,961 task examples, split into 2,095 for training, 424 for validation and 442 for testing. Figure 5.1 shows the spread across domains. Software and technical projects is the biggest with 240 rows and the rest have roughly 110 to 140 each. (Several tasks can come from one episode, so this counts rows, not conversations.)

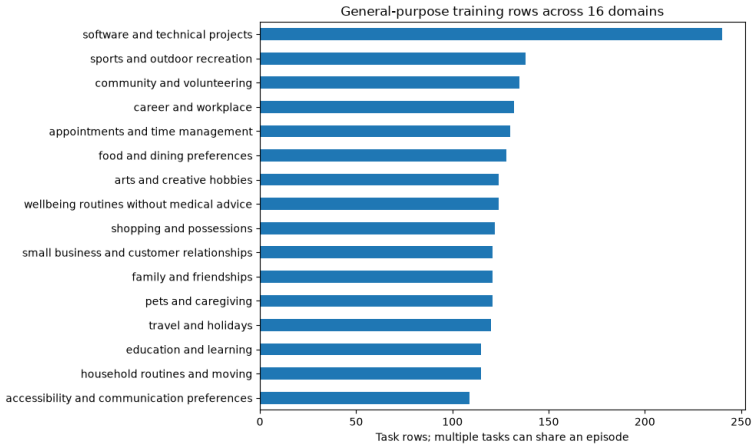


Figure 5.1: General-purpose training rows across the 16 domains

A lot of the rows test whether the model keeps only what the user actually said. For example, the assistant says "You could learn violin this winter." and the user replies "No, that is not my plan. I am learning watercolor painting." The expected memory is just that the user is learning watercolor painting. The assistant's suggestion should not become a memory.

For testing outside our own data distribution we used LoCoMo [15], which has real-style conversations over many sessions. We took 16 conversation segments from 10 conversations, with 40 reference facts. A typical one is Caroline and Melanie chatting about a pottery class and a painting; the expected memory is that Melanie made a plate in pottery class and Caroline painted a picture inspired by her beach visit.

5.2 FINE-TUNING AND MODEL SELECTION

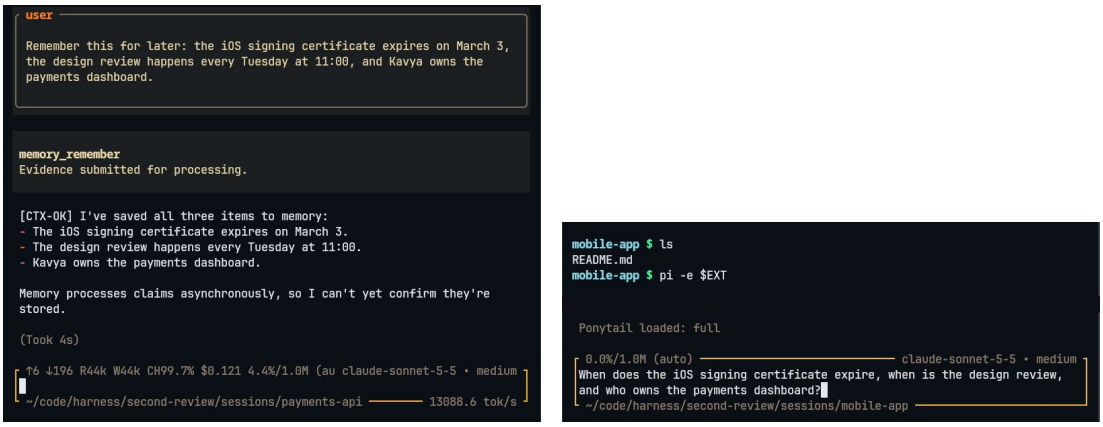
The model is Qwen3-4B-Instruct-2507, fine-tuned with QLoRA [14] and served through Ollama. Following Figure 3.6, GPT-5.6-sol labelled the episodes for the extract, consolidate and reflect tasks, every label was checked for structure and evidence (and repaired if needed), and the valid ones were

frozen into the 2,095/424/442 split. Rows were tokenised with a 4,096 token limit and the prompt was masked so only the target is supervised. Training used QLoRA on the frozen base with NF4 quantisation and rank 16, going from Stage A to Stage B. The adapter was then merged and quantised to a Q8_0 GGUF with llama.cpp, compared against the base model on 64 internal tasks and 16 external cases, checked live (retain, graph, recall/reflect), and packaged with a SHA-256 checksum and imported into Ollama as `mem-extractor:general-v1`.

We actually started with Qwen3-1.7B. It was fine on synthetic examples but bad on natural conversations, only 18 of 33 extraction windows passed validation even after refinement. Most failures were wrong source attribution, quotes that didn't match and invalid IDs. That's what pushed us to broaden the training data, tighten the output constraints and move up to the 4B model.

5.3 LIVE DEMONSTRATION

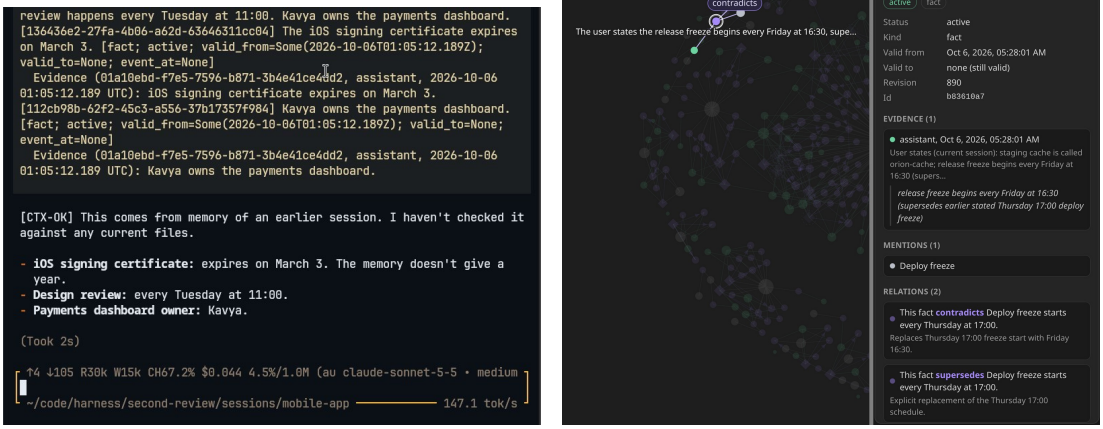
The demo follows the same idea as Maya (record, ask later in a new session, see the correction) but with its own values: a certificate expiry date, a weekly review time, a dashboard owner, and later a release-freeze time. Figure 5.2 shows the first two steps.



(a) Recording in the first session (b) Asking in a new session

Figure 5.2: (a) The user gives a certificate expiry date, a weekly review time and the dashboard owner, which are submitted for memory processing. (b) A fresh session in another project directory asks for the same details without repeating them

In (a) the agent calls the `memory_remember` tool and notes that processing is asynchronous, so it can't confirm storage yet. In (b) a new session in `mobile-app` (not `payments-api`) asks for all three without restating them, like Maya asking E5.



(a) Recalled answer (b) Corrected fact in the graph

Figure 5.3: (a) Memory from the first session is added to the context and the agent answers March 3, Tuesday at 11:00 and Kavya. (b) The graph shows Friday at 16:30 as the current release-freeze time, linked to the previous Thursday at 17:00 value, with the supporting message and history in the side panel

In Figure 5.3(a) the agent answers from the recalled facts (March 3, since no year was given, then Tuesday 11:00 and Kavya) and says the information came from an earlier session and wasn't checked against the current files. In (b) the Friday 16:30 freeze is linked to the old Thursday 17:00 value by `contradicts` and `supersedes`, same as Bengaluru and Chennai.

5.4 RESULTS AND OBSERVATIONS

Most of what we know so far comes from the demos, a database audit and the model comparison.

In a three-session coding demo the required output changed from milliseconds to microseconds partway through. In the next session the system passed this on to the agent without the user repeating it, and the agent used

microseconds. In another test conversation we gave a person's city and food preference and then changed only the city. The system used the new city, kept the old one as history, and still had the food preference.

The database audit found that all 142 stored source quotes matched their original text exactly, though a matching quote doesn't prove the model understood it correctly. It also found 8 observations, each linked to at least two supporting facts. Dates are still weak: a message sent on June 1 said "tomorrow" and the system read it as June 3.

Table 5.1 compares Qwen3-4B before and after fine-tuning, on 64 unseen tasks across the 16 domains and on 16 external conversation samples.

Table 5.1: Qwen3-4B before and after fine-tuning

Measurement	Before	After
Correctly combined related facts	1/16	13/16
Correct answers using supplied evidence	10/16	15/16
Expected facts recovered	50/55	41/55
Extracted claims supported by the source	92.1%	90.5%
Corrections handled correctly	8/8	8/8
Outputs passing format checks	64/64	63/64
Expected facts recovered (16 external samples)	16/40	13/40

Combining facts and answering clearly got better (1/16 to 13/16 and 10/16 to 15/16), and corrections were right in all 8 cases both times. But extraction got worse. Facts recovered dropped from 50/55 to 41/55 internally and 16/40 to 13/40 on the external samples, supported claims dipped a bit from 92.1% to 90.5%, and one output failed the format check. Honestly this was a bit disappointing and we haven't fully figured out why yet. Passing the format check also says nothing about whether the facts are right (the June 1 date passed it). Since the full benchmark isn't done, we can't claim an overall improvement

on long conversation histories yet.

5.5 STATUS AND PENDING WORK

M1 is implemented and covered by the Pi integration tests (capture, retries, namespace changes). M2 is implemented but its extraction quality still needs work. M3 is implemented, with integration tests for validity intervals, conflicts, invalidation and graph cleanup, but needs testing on harder updates. M4 is implemented and has retrieval tests and live checks, but the full benchmark is pending. For M5, training and integration are done and evaluation is still going on. The pending items for each module are listed in the next section.

5.6 CONCLUSION AND FUTURE WORK

We built all five modules of the memory engine: evidence capture, fact extraction and consolidation, temporal graph management, hybrid retrieval with grounded answers, and model training. Four are implemented and M5 is trained and integrated with evaluation still going on. In the demos and audit, information from one session was recalled in a new one, a corrected fact was stored with its history without touching unrelated facts, and all 142 source quotes matched. Fine-tuning helped a lot with combining facts (1/16 to 13/16) and answering from evidence (10/16 to 15/16).

Some things aren't there yet. Extraction regressed after fine-tuning (50/55 to 41/55 internal, 16/40 to 13/40 external), relative dates are sometimes wrong, and without the full LongMemEval-S run we can't say it's better over long histories. Future work, by module:

- M1: API access control and verified backup and restore.

- M2: better missed-fact coverage, date interpretation and source attribution; measure the review step's quality and latency.
- M3: evaluate ambiguous updates, aliases and longer correction histories.
- M4: finish LongMemEval-S, compare graph and consolidation ablations, report retrieval quality and latency.
- M5: fix the extraction regression and date errors, more external testing and independent human review.

REFERENCES

[1] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Trans. Assoc. Comput. Linguist.*, vol. 12, pp. 157–173, 2024. [Online]. Available: https://aclanthology.org/2024.tacl-1.9/

[2] D. Wu, H. Wang, W. Yu, Y. Zhang, K.-W. Chang, and D. Yu, "LongMemEval: Benchmarking chat assistants on long-term interactive memory," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2025, pp. 86 809–86 836, also arXiv:2410.10813. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2025/hash/d813d324dbf0598bbdc9c8e79740ed01-Abstract-Conference.html

[3] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems," arXiv preprint arXiv:2310.08560, Oct. 2023, (rev. Feb. 2024). [Online]. Available: https://arxiv.org/abs/2310.08560

[4] P. Rasmussen, P. Paliychuk, T. Beauvais, J. Ryan, and D. Chalef, "Zep: A temporal knowledge graph architecture for agent memory," arXiv preprint arXiv:2501.13956 [cs.CL], Jan. 2025. [Online]. Available: https://arxiv.org/abs/2501.13956

[5] P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav, "Mem0: Building production-ready AI agents with scalable long-term memory," in *Proc. 28th Eur. Conf. Artif. Intell. (ECAI)*, ser. Frontiers in Artificial Intelligence and Applications, vol. 413. IOS Press, Oct. 2025, pp. 2993–3000, also arXiv:2504.19413 [cs.CL].

[6] S. Daga, S. Sreedhar, and D. Shah, "LongMemEval," supermemory research (vendor-hosted evaluation report on LongMemEval-S, not peer reviewed), May 2026, (accessed Oct. 6, 2026). [Online]. Available: https://supermemory.ai/research/longmembench/

[7] C. Latimer, N. Boschi, A. Neeser, C. Bartholomew, G. Srivastava, X. Wang, and N. Ramakrishnan, "Hindsight is 20/20: Building agent memory that retains, recalls, and reflects," arXiv preprint arXiv:2512.12818 [cs.CL], Dec. 2025. [Online]. Available: https://arxiv.org/abs/2512.12818

[8] Neo4j, Inc., "Query the database," in *Neo4j HTTP API Manual, Version 5* (page marked deprecated in 5.26). Neo4j, Inc., 2026, (accessed Oct. 6, 2026). [Online]. Available: https://neo4j.com/docs/http-api/5/query/

[9] M. Zechner *et al.*, "pi: AI agent toolkit (unified LLM API, agent loop, TUI, coding agent CLI)," package @earendil-works/pi-coding-agent, v1.0.2 (version locked by the project's package-lock.json; latest is 1.0.4 on Oct. 6, 2026), GitHub repository, 2026, (accessed Oct. 6, 2026). [Online]. Available: https://github.com/earendil-works/pi

[10] PostgreSQL Global Development Group, "PostgreSQL 17 documentation, chapter 12: Full text search (sections 12.2.2, 12.3.2, 12.3.3, 12.9) and section 64.4: GIN indexes," PostgreSQL 17.11 documentation, 2026, (accessed Oct. 6, 2026). [Online]. Available: https://www.postgresql.org/docs/17/textsearch.html

[11] A. Kane *et al.*, "pgvector: Open-source vector similarity search for Postgres," source code repository and README (0.8.6 installed in the test database, v0.8.7 tag also released), Oct. 2026, (accessed Oct. 6, 2026). [Online]. Available: https://github.com/pgvector/pgvector

[12] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. 32nd Int. ACM SIGIR Conf. Research and Development in Information Retrieval (SIGIR '09)*, Boston, MA, USA, Jul. 2009, pp. 758–759.

[13] Y. Zhang, M. Li, D. Long, X. Zhang, H. Lin, B. Yang, P. Xie, A. Yang, D. Liu, J. Lin, F. Huang, and J. Zhou, "Qwen3 Embedding: Advancing text embedding and reranking through foundation models," arXiv preprint arXiv:2506.05176 [cs.CL], Jun. 2025. [Online]. Available: https://arxiv.org/abs/2506.05176

[14] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023, pp. 10 088–10 115, also arXiv:2305.14314.

[15] A. Maharana, D.-H. Lee, S. Tulyakov, M. Bansal, F. Barbieri, and Y. Fang, "Evaluating very long-term conversational memory of LLM agents," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (Volume 1: Long Papers)*, Bangkok, Thailand, Aug. 2024, pp. 13 851–13 870, (preprint: arXiv:2402.17753). [Online]. Available: https://aclanthology.org/2024.acl-long.747/